CMPS 4191 - Advanced Web Technologies - Quiz 2

Asynchronous Job + Worker + HTTP 202

Reynerio Samos - 2018119235
August 30, 2026

Slides:

- The problem requirement
- HTTP contract
- Architecture
- Job lifecycle
- Job Create and Persistence
- Claiming jobs safely
- Worker behavior
- Report SQL and Outcomes
- Graceful shutdown
- Measurement and Interpretation

The Problem and Requirement

Synchronous vs Asynchronous

Synchronous

- Blocking
- Timeout risk
- Bad client experience
- Underutilized resource

Asynchronous

- Nonblocking
- Persistent
- Better client experience
- Better use of resource

Requirement Change:

“POST no longer promises completed report, Only to acknowledge work and return job identity”

The need to acknowledge work quickly is now needed as report generation is decoupled from the response. This allows for background workers, status and progress tracking on jobs.

Using a HTTP 202 Accepted for immediate acknowledgement is ideal.

HTTP Contract

POST /v1/reports And 202 Accepted

Request

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2026-02-01T00:00:00Z"
}
```

Response

```
{
 "job_id": "1198f000-0000-4000-8000-000000000201",
  "status": "queued",
  "status_url": "/v1/jobs/1198f000-0000-4000-8000-000000000201"
}
```

HTTP 202 Accepted:

Means

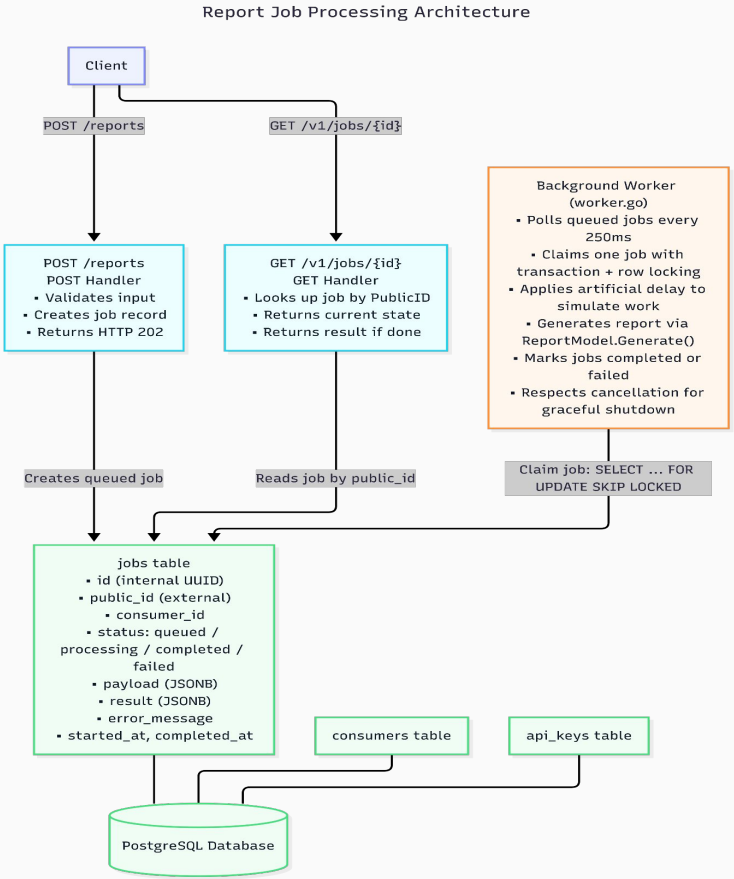
- Request accepted for processing
- Provides location to check status

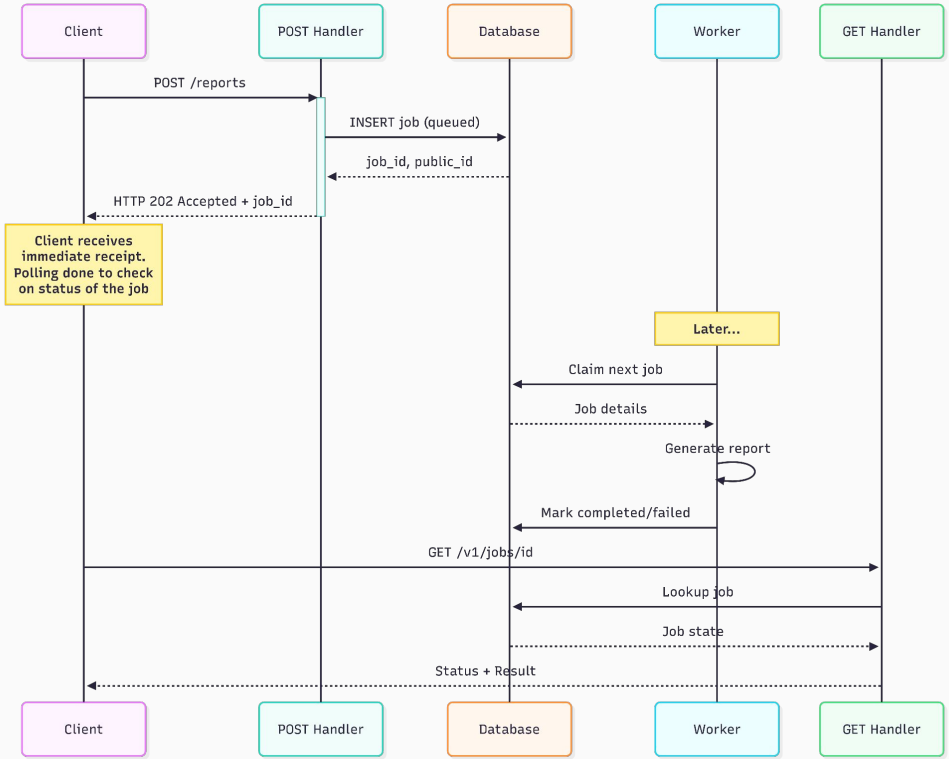
Does not mean

- Guarantee of Success
- Final Result

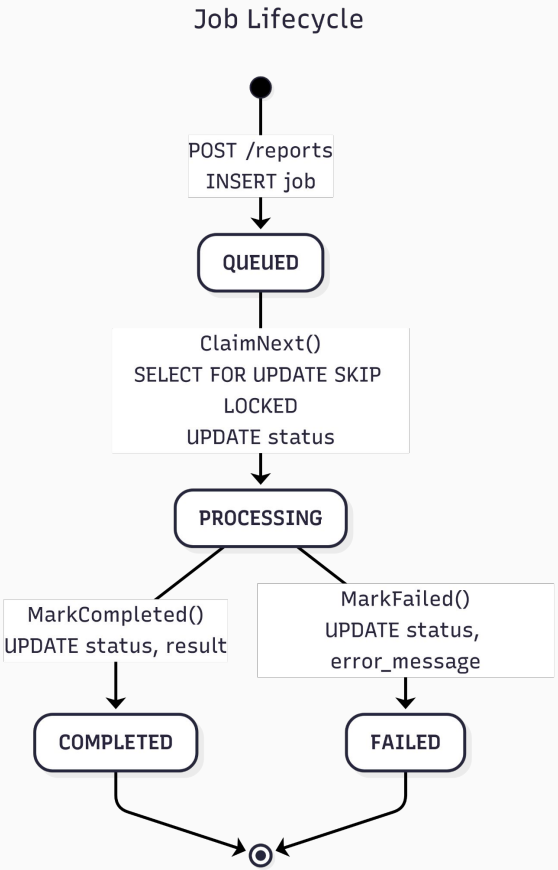
GET /v1/jobs/{id} used to query job state as a read only operation. Returns current status and results if finished

System Architecture





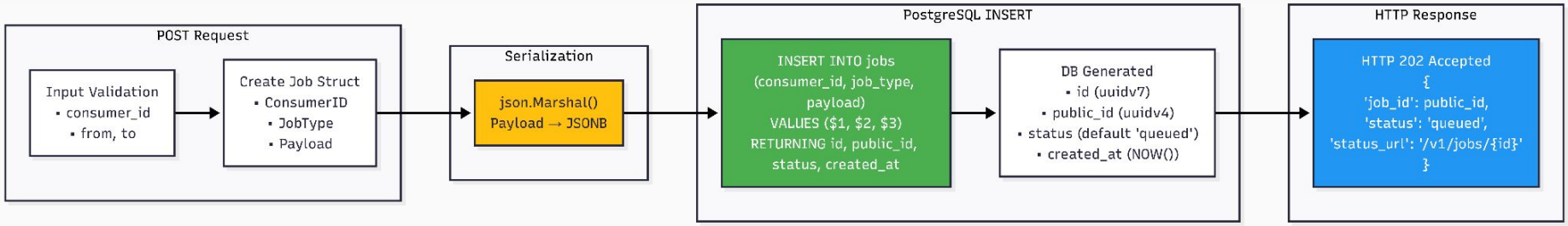
Job Life Cycle



Transitions, Functions and DB Changes:

Transition		Function	Database Change
Queued	Processing	JobModel.ClaimNext	status='processing', started_at=NOW()
Processing	Completed	JobModel.MarkComplete	status='completed', result=JSON, completed_at=NOW()
Processing	Failed	JobModel.MarkFailed	status='failed', error_message=TEXT, completed_at=NOW()

Job Creation and Persistence



Before Job is accepted

- Input is validated
- Job struct is made

Then the struct is sent as a serialized JSOB from a JSON using json.Marshall

JSOB just has better indexing and is more optimized for database operations

Job is then inserted into the jobs database and a public_id, status and timestamp created is returned back to the HTTP handler for the POST request with a 202 accepted, finishing the contract.

Claiming Jobs Safely

```sql
BEGIN TRANSACTION;

-- Select one queued job, lock it, skip already-locked rows

SELECT id, public_id, consumer_id, job_type, payload

FROM jobs

WHERE status = 'queued' AND job_type =
'consumer_activity_report'

ORDER BY created_at  -- FIFO ordering

FOR UPDATE SKIP LOCKED  -- Prevent double-claiming

LIMIT 1;  -- One worker, one job

-- Immediately mark as processing in same transaction

UPDATE jobs

SET status = 'processing', started_at = NOW()

WHERE id = $1;

COMMIT;
```

Transaction is an atomic operation, making claiming and marking one coupled step to ensure no two workers claim the same job

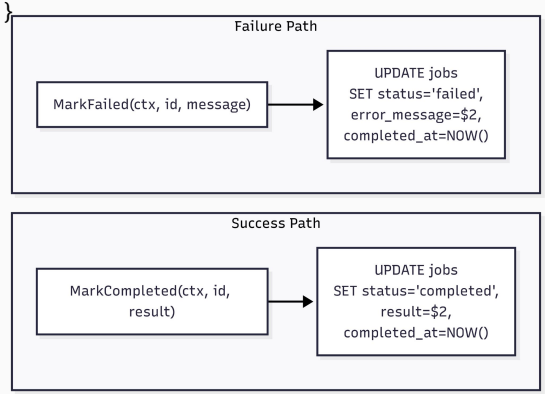
FOR UPDATE SKIP LOCKED is used by workers to ensure they don't try to claim a job that is already being worked on

ORDER BY created_at gives oldest jobs priority in being chosen (queue enforces FIFO)

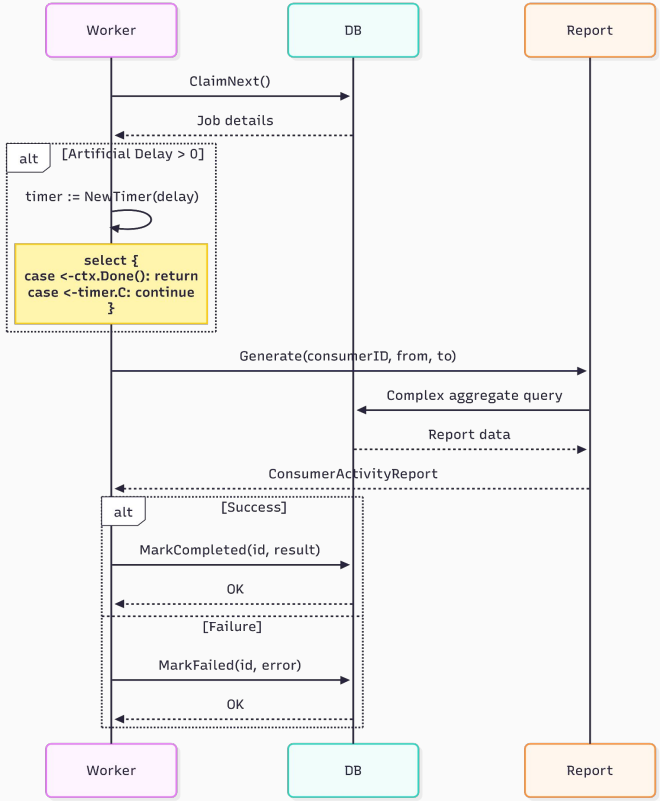
LIMIT 1 limits a worker to one job, so while the API itself is asynchronous, the workers aren't and cannot work on more than one job at a time or do anything else while working in that job

Worker Behavior

```go
func (app *application) startReportWorker(ctx context.Context) {
    app.wg.Add(1)  // Track goroutine for shutdown

    go func() {
        defer app.wg.Done()  // Signal when done

        ticker := time.NewTicker(app.config.workerPollInterval)
// 250ms

        for {
            select {
            case <-ctx.Done():
                app.logger.Info("report worker stopped")
                return  // Graceful shutdown
            case <-ticker.C:
                app.processNextReportJob(ctx)  // Process one job
            }
        }
    }()
}
```



Sequence Diagram for a single worker



Report SQL and outcomes

SQL

```sql
SELECT c.id, c.name,
    COUNT(DISTINCT k.id) FILTER (WHERE k.status = 'active'),
    COUNT(DISTINCT k.id) FILTER (WHERE k.status = 'revoked'),
    COUNT(DISTINCT j.id) FILTER (WHERE j.status = 'queued'),
    COUNT(DISTINCT j.id) FILTER (WHERE j.status = 'processing'),
    COUNT(DISTINCT j.id) FILTER (WHERE j.status = 'completed'),
    COUNT(DISTINCT j.id) FILTER (WHERE j.status = 'failed')
FROM consumers c
LEFT JOIN api_keys k ON k.consumer_id = c.id
LEFT JOIN jobs j ON j.consumer_id = c.id
    AND j.created_at >= $2 AND j.created_at < $3
WHERE c.id = $1
GROUP BY c.id, c.name
```

- JOIN is used to join different tables together using a common key (in this case consumer_id)
- FILTER(WHERE) is used for conditional counting within the aggregation. In this case the condition is based on status
- COUNT(DISTINCT) used to prevent multiples of the same record being counted (in this case the joins make a lot of duplicates)
- GROUP BY is used to show one row per consumer based on their id and name

Recording Outcomes

Success

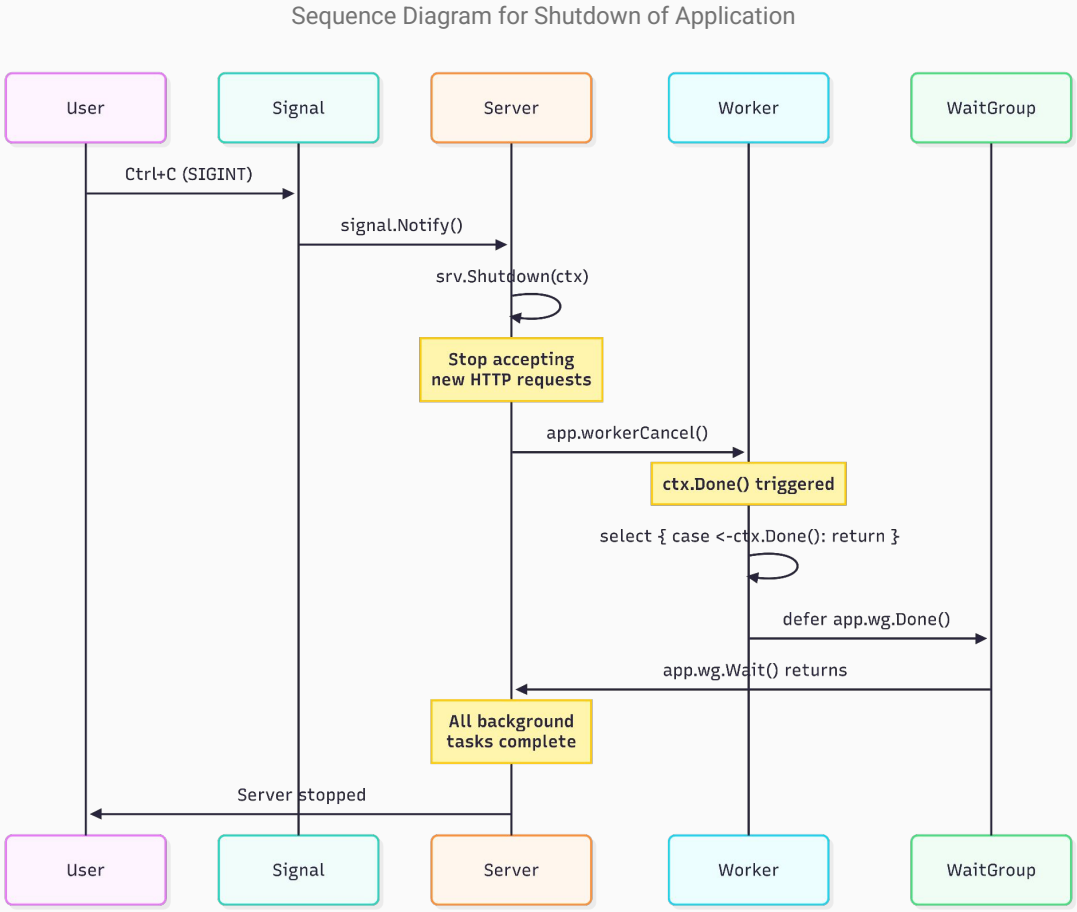
```go
func MarkCompleted(ctx, id, result []byte) {
    UPDATE jobs
    SET status = 'completed', result = $2,
completed_at = NOW()
    WHERE id = $1
}
```

Failure

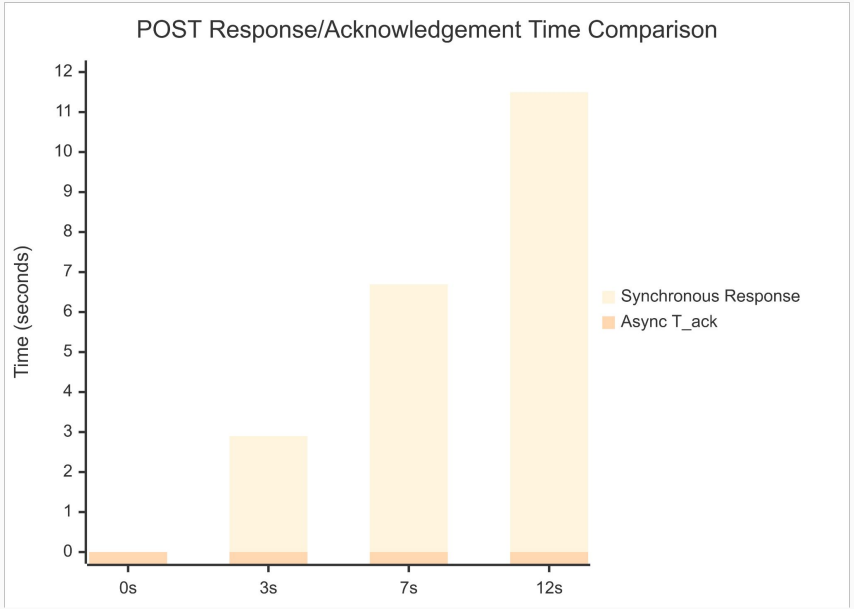
```go
func MarkFailed(ctx, id, message string) {
    UPDATE jobs
    SET status = 'failed', error_message = $2,
completed_at = NOW()
    WHERE id = $1
}
```

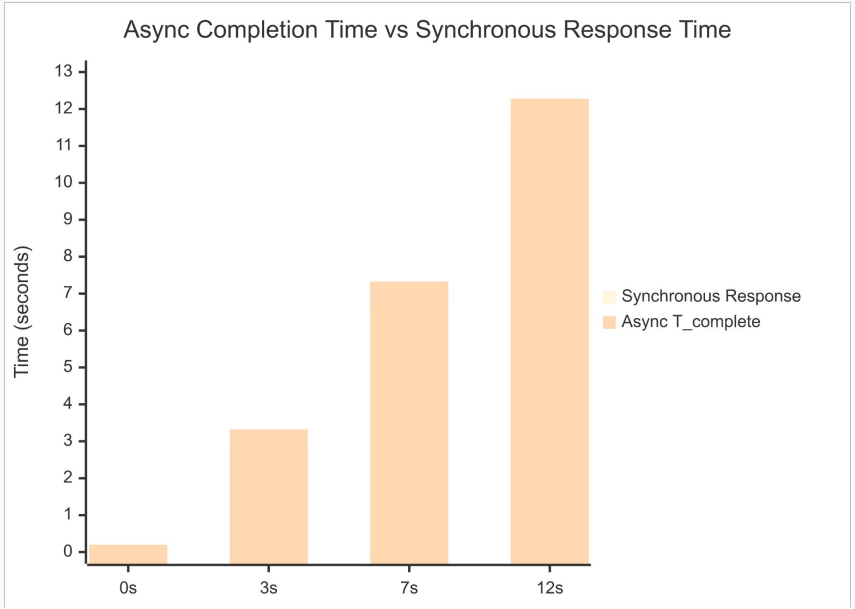
Graceful shutdown

```
    go func() {
        quit := make(chan os.Signal, 1)
        signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
        s := <-quit

        app.logger.Info("caught signal", "signal", s.String())

        // gives 30 seconds for a HTTP request to finish before
being forcefully closed
        // used for finishing remain jobs during shutdown proccess
        ctx, cancel := context.WithTimeout(context.Background(),
30*time.Second)
        defer cancel()

        // stops new HTTP connections from being created and
interrupts the inflight ones base on time
        err := srv.Shutdown(ctx)
        if err != nil {
            shutdownError <- err
            return
        }

        app.logger.Info("completing background tasks", "addr",
srv.Addr)


        if app.workerCancel != nil {
            app.workerCancel()
        }
        // blocks until worker goroutine is started
        app.wg.Wait()
        shutdownError <- nil
    }()
```



Measurement and Interpretation





For acknowledgement, the async API had a consistent time while the sync API had increasing acknowledgement time to report delay

However, the time for the report to be finished is still the same, the only thing that changes is that the user has acknowledgement before it's done.

The worker cannot mark the job as done until the artificial delay expires, making T_complete still dependent on the delay.